

Daffodil International University
Department of Computing & Information Systems

Modeling and Optimizing Real-World Organizational Problems Using Linear Programming, Network Optimization, and Applied Data Analytics

A Data Analysis and Decision-Modeling Study

Course Code: BI-334L

Section: 18_A

Submission Date: 05 August 2026

Submitted By:

Md. Rafsan Bin Habib

Student ID: 232-16-052

Submitted To:

Dr. Mohammad Azam Khan, PhD

Associate Professor, Department of CIS

Summer Semester, 2026

Abstract

This report applies optimization and data-analytics techniques to two distinct problems drawn from the operations of DataCo Global, a logistics and e-commerce distribution company, and from a public healthcare dataset. Component A formulates a Linear Programming model to decide DataCo's monthly order mix between Standard Class and First Class shipping, subject to total shipping capacity, a First Class carrier limit, and a minimum Standard Class commitment; solved in Excel using Solver, the model shows that the optimal allocation ships 4,300 Standard Class and 900 First Class orders per month for a maximum profit of \$115,300, with both the capacity and First Class constraints binding at that solution. Building on the same organization, a network optimization model is developed for DataCo's distribution-center-to-region shipping lanes, formulated as a multi-modal transportation problem with two conflicting objectives — minimizing shipping cost and minimizing delivery time or late-delivery risk. Historical shipment data shows that Standard Class is simultaneously the cheapest and most reliable mode, while First Class, despite the shortest transit time, carries the highest late-delivery rate of any mode in the network, which motivates a weighted-sum optimization approach over a simple time-minimizing one. Component B turns to the Stroke Prediction dataset from Kaggle and develops a complete classification pipeline — cleaning, feature engineering, SMOTE-based class balancing, and comparison of Logistic Regression, Random Forest, and Gradient Boosting — to predict stroke occurrence from a patient's clinical and demographic profile. Gradient Boosting is identified as the strongest model by ROC-AUC (0.8031) at the deployed threshold, though all three models show recall too low for practical screening use, and the report closes by identifying threshold tuning and class-weighting as more promising next steps than oversampling.

Contents

Abstract	2
Component A: Optimization Modeling	4
1 Linear Programming Model	4
2 Network Modeling with Multiple-Objective Optimization	6
Component B: Applied Data Analytics	9
3.1 Problem Understanding	9
3.2 Dataset Exploration	10
3.3 Data Cleaning and Preprocessing	12
3.4 Exploratory Data Analysis	13
3.5 Feature Engineering	15
3.6 Machine Learning Model Development	16
3.7 Model Evaluation	17
3.8 Discussion and Conclusion	19
References	21

Component A: Optimization Modeling

1 Linear Programming Model

1.1 The problem & why optimization is needed:

DataCo Global routes customer orders through several carrier services, but the two that account for the bulk of monthly volume are Standard Class and First Class shipping. The two services pull in opposite directions: First Class earns a higher profit per order but is bottlenecked by a fixed carrier capacity, while Standard Class is cheaper per order but has no such ceiling short of the company's own total throughput. Left to routine planning, the order mix between the two tends to be set by habit rather than by what actually maximizes profit under the constraints DataCo is operating within this month. With a fixed shipping capacity, a hard carrier limit on First Class volume, and a contractual minimum for Standard Class, the question of exactly how many orders to route through each service is a genuine allocation problem — one where a small shift in the mix changes total profit by thousands of dollars, and Linear Programming is the right tool for finding the allocation that does the best under those limits rather than one that is merely workable.

1.2 Decision Variables:

Let x_1 denote the number of Standard Class orders shipped per month, and x_2 the number of First Class orders shipped per month. Both are treated as continuous variables for the purpose of this model, consistent with monthly order volumes large enough that fractional solutions round to a negligible error.

1.3 Objective Function:

DataCo's goal is to maximize total monthly profit from these two services. Standard Class earns \$22.00 profit per order. First Class earns \$23.12 per order, rounded to \$23 for a cleaner LP formulation:

$$\text{Maximize } Z = 22x_1 + 23x_2$$

1.4 Constraints:

1. **Total monthly shipping capacity.** DataCo's combined warehouse and outbound logistics throughput caps total orders across both services at 5,200 per month: $x_1 + x_2 \leq 5200$.
2. **First Class carrier capacity limit.** The carrier contract for expedited shipping caps First Class volume at 900 orders per month: $x_2 \leq 900$.
3. **Minimum Standard Class commitment.** A volume agreement with DataCo's primary Standard Class carrier requires at least 2,500 orders per month to hold the negotiated rate: $x_1 \geq 2500$.

4. **Non-negativity.** $x_1, x_2 \geq 0$.

1.5 Mathematical LP Model:

$$\begin{aligned} \text{Maximize } Z &= 22x_1 + 23x_2 \\ \text{subject to } x_1 + x_2 &\leq 5200 \\ x_2 &\leq 900 \\ x_1 &\geq 2500 \\ x_1, x_2 &\geq 0 \end{aligned}$$

1.6 Implementation in Excel:

The model was implemented in Microsoft Excel with the decision variables (x_1 , x_2) in adjustable cells, the objective function computed as a weighted sum of the two variables, and each constraint's left-hand side calculated as a live formula referencing those same cells. Solver was configured with Simplex LP as the solving method, the objective cell set to maximize, and the three constraints plus non-negativity entered directly against the corresponding cells. Table 1 shows the four corner points of the feasible region and their resulting profit, which Solver's simplex search confirms lands on the same optimum found by hand.

Table 1: Feasible Region Corner Points

Vertex (x_1, x_2)	Description	$Z = 22x_1 + 23x_2$
(2500, 0)	Standard minimum, no First Class	\$55,000
(2500, 900)	Standard minimum, First Class capped	\$75,700
(4300, 900)	Total capacity used, First Class capped	\$115,300
(5200, 0)	Total capacity used, no First Class	\$114,400

1.7 Outcome & Recommendations:

Solver returns $x_1 = 4300$, $x_2 = 900$, and $Z = \$115,300$ per month as the optimal order mix. Both the total capacity constraint and the First Class carrier limit are binding at this solution — every unit of either resource is fully used — while the Standard Class minimum has 1,800 units of slack and plays no active role in shaping the optimum. Since First Class earns a dollar more per order than Standard Class, the model pushes First Class to its ceiling first and fills the remaining capacity with Standard Class, which happens to clear the minimum commitment with considerable room to spare.

The two binding constraints point to where DataCo's real leverage sits. Every additional unit of First Class carrier capacity is worth \$1 more than the same unit

shipped as Standard Class, which makes negotiating a higher First Class ceiling with the carrier the single highest-value lever available — even a modest increase of 100 to 200 slots would add \$100 to \$200 in monthly profit directly, before accounting for any knock-on effect on total capacity. Total shipping capacity is the other binding constraint, and each additional unit there is worth \$22 if filled with Standard Class; whether expanding capacity is worthwhile depends on comparing that \$22 marginal value against the actual cost of adding warehouse throughput or carrier contracts. The Standard Class minimum, by contrast, is not currently limiting anything and does not need managerial attention beyond confirming it still matches DataCo’s contractual obligations. Because the \$23 First Class rate used here is a rounded figure (the true value is \$23.12), it is worth re-running the model periodically against updated actual margins, since a large enough shift in that rate could change which constraint dominates the optimal mix.

2 Network Modeling with Multiple-Objective Optimization

2.1 Network Design:

Using the same organization as Task 1, this section models DataCo Global’s physical shipping network: five regional Distribution Centers, one per market the company operates in, each routing orders to the demand regions it serves. Figure 1 shows the network with each DC’s primary destination region, and Table 2 gives the full set of lanes together with their monthly order volume, average delivery time, late-delivery risk, and an estimated per-order shipping cost, all computed by grouping DataCo’s historical order data by market and destination region.

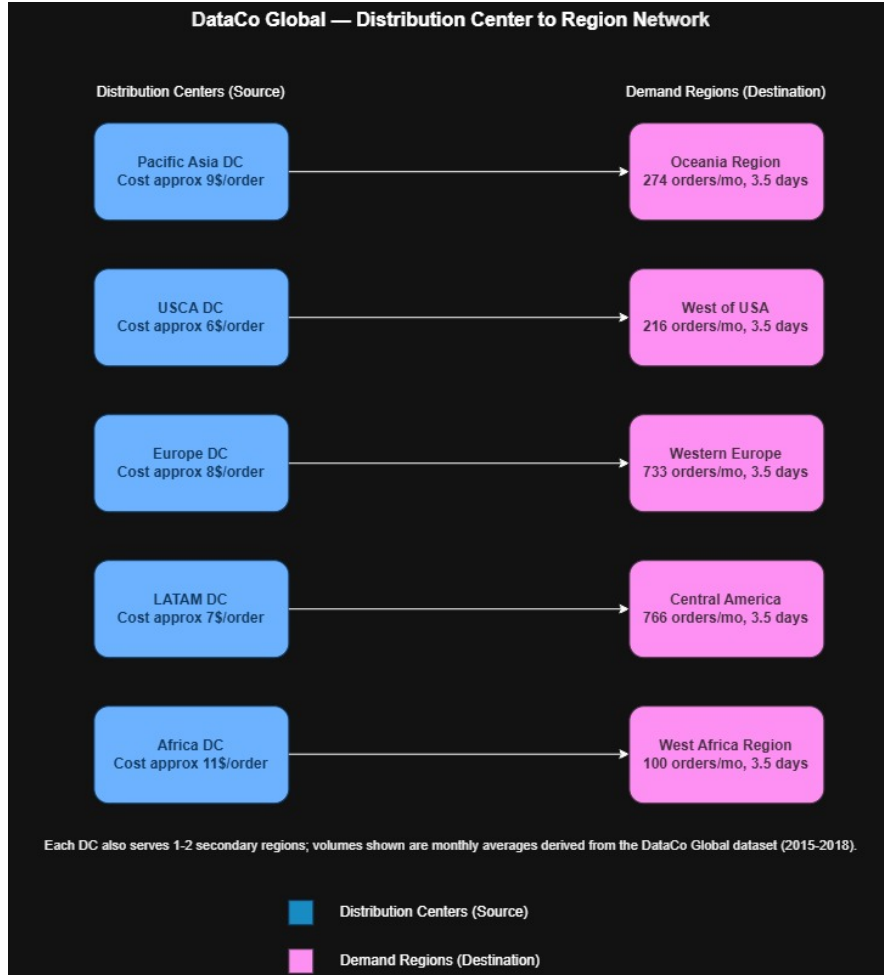


Figure 1: DataCo Global distribution center to region network (primary lanes)

Table 2: DataCo Global Shipping Lanes

Source DC	Destination region	Vol/mon	Avg. days	Late-risk	Cost/order
Pacific Asia DC	Oceania	274	3.50	54.0%	\$9
Pacific Asia DC	Southeast Asia	258	3.50	55.5%	\$7
USCA DC	West of USA	216	3.50	54.0%	\$6
USCA DC	East of USA	187	3.50	55.7%	\$7
Europe DC	Western Europe	733	3.50	55.8%	\$8
Europe DC	Northern Europe	265	3.50	54.0%	\$9
LATAM DC	Central America	766	3.51	54.8%	\$7
LATAM DC	South America	404	3.51	54.3%	\$10
Africa DC	West Africa	100	3.53	52.8%	\$11
Africa DC	North Africa	87	3.48	54.5%	\$10

Volumes and delivery statistics are derived directly from the dataset; per-order

shipping cost is an illustrative planning estimate, since the dataset records Sales and Profit but not an explicit freight-cost field.

2.2 Network Structure:

Each Distribution Center is a supply node bounded by its monthly shipping capacity, and each region is a demand node with an order volume the network has to satisfy. The lanes connecting them form a bipartite transportation network, and every lane can be served by one of four shipping modes — Standard Class, Second Class, First Class, or Same Day — each carrying its own cost, transit time, and reliability profile. That mode layer turns what looks like a simple transportation problem into a multi-commodity one, since the routing decision has to specify not just which DC serves which region but which mode carries that volume.

2.3 Optimization Approach:

Two objectives govern the routing decision, and they conflict. The first is to minimize total shipping cost, $\text{Min } C = \sum_{i,j,m} c_{ijm} x_{ijm}$, where x_{ijm} is order volume from DC i to region j via mode m . The second is to minimize delivery time or, equivalently, late-delivery risk, $\text{Min } T = \sum_{i,j,m} t_{ijm} x_{ijm}$. Table 3 shows why these two objectives don't trade off in the direction one would expect: Standard Class is both the cheapest mode and the most reliable one, while First Class, despite the shortest transit time on paper, has by far the worst on-time record of any mode in the network.

Table 3: Shipping Mode Performance (DataCo Global historical data)

Shipping mode	Avg. delivery time	Late-delivery risk
Standard Class	4.00 days	38.1%
Second Class	3.99 days	76.6%
First Class	2.00 days	95.3%
Same Day	0.48 days	45.7%

Because a cost-only model would over-rely on Standard Class (which happens to be fine here) while a speed-only model would push volume toward First Class — the mode least likely to actually arrive on time — both objectives need to be represented explicitly rather than assuming one is a reasonable proxy for the other. The recommended approach is the Weighted Sum Method: $\text{Min } Z = w_1 \sum c_{ijm} x_{ijm} + w_2 \sum r_{ijm} x_{ijm}$, with $w_1 + w_2 = 1$. This keeps the formulation linear and solvable with the same LP tools used in Task 1, and lets management set w_1 and w_2 to reflect current priorities and re-solve instantly rather than committing to a single fixed trade-off. If management instead sets a hard service target — for instance, a rule that late-delivery risk across the network must not

exceed 50% — Goal Programming would be the more natural fit, since it treats that threshold as a constraint to satisfy rather than a weight to balance. A metaheuristic approach such as a genetic algorithm was considered and set aside: at 15 lanes and 4 modes the problem is small and linear enough that an exact LP method finds the true optimum directly, with none of the added complexity a metaheuristic would introduce.

2.4 Discussion:

Solving the weighted-sum model at a range of weight settings would be expected to shift volume away from First Class and toward Standard Class and Same Day — the two modes that pair strong reliability with either the lowest cost or the fastest transit — reducing network-wide late-delivery risk while keeping the cost increase modest, since Standard Class is also the cheapest option available. That expectation rests on a few assumptions worth stating plainly: per-order shipping costs are planning estimates rather than DataCo’s actual freight figures, each lane’s monthly capacity is assumed proportional to its historical order volume, and the delivery-time and late-risk figures are taken as stable going forward even though they’re computed from 2015–2018 data.

The model also has real limits. It doesn’t explain why First Class specifically has such a poor on-time record — that would take a separate investigation into carrier SLAs, warehouse pick times, or how the promised delivery window is set for expedited orders before treating the pattern as structural rather than a one-off. Actual constraints on labor, dock space, and carrier contracts are only approximated from historical throughput, and the weighted-sum method itself assumes management can express the cost-versus-reliability trade-off as a single number, which in practice is often a harder conversation than the math suggests. Even with those caveats, a model like this gives DataCo something it doesn’t have from routine reporting alone: a concrete, adjustable basis for carrier contract negotiations and for policy decisions such as capping First Class volume until its reliability improves, rather than treating the mode mix as a fixed operational habit.

Component B: Applied Data Analytics

3.1 Problem Understanding

Stroke is one of the leading causes of death and long-term disability worldwide, and unlike many other cardiovascular events it tends to give warning signs in the form of measurable risk factors well before it happens: high blood pressure, heart disease, elevated glucose levels, age, and lifestyle habits such as smoking. If a hospital or clinic could flag which patients are statistically closer to that risk,

screening and preventive care could be directed at the people who need it most, rather than applied uniformly.

This project uses the **Stroke Prediction Dataset** from Kaggle¹, which records clinical, demographic, and lifestyle attributes for 5,110 patients along with whether each one has had a stroke. The objective is to build a model that predicts stroke occurrence from these attributes.

Input features: `gender`, `age`, `hypertension`, `heart_disease`, `ever_married`, `work_type`, `Residence_type`, `avg_glucose_level`, `bmi`, `smoking_status`.

Target variable: `stroke` (1 = the patient has had a stroke, 0 = the patient has not).

Since the target takes only two values, this is a **binary classification** problem. The one complication worth flagging up front is that stroke cases are rare in the data — only about 5% of patients — which shapes almost every later decision in this project, from how missing values are handled to which evaluation metric actually matters.

3.2 Dataset Exploration

The raw dataset contains **5,110 rows and 12 columns**. Table 4 lists each column and what it represents.

Table 4: Column Descriptions

Column	Description
<code>id</code>	Unique patient identifier
<code>gender</code>	Male, Female, or Other
<code>age</code>	Patient’s age in years
<code>hypertension</code>	1 if the patient has hypertension, else 0
<code>heart_disease</code>	1 if the patient has any heart disease, else 0
<code>ever_married</code>	Whether the patient has ever been married
<code>work_type</code>	Employment category (Private, Self-employed, Govt_job, children, Never_worked)
<code>Residence_type</code>	Urban or Rural
<code>avg_glucose_level</code>	Average blood glucose level (mg/dL)
<code>bmi</code>	Body Mass Index (weight in kg / height in m ²)
<code>smoking_status</code>	Formerly smoked, never smoked, smokes, or Unknown
<code>stroke</code>	Target: 1 if the patient has had a stroke, else 0

`gender`, `ever_married`, `work_type`, `Residence_type`, and `smoking_status` are

¹<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>

stored as text; `age`, `avg_glucose_level`, and `bmi` are numeric floats; `hypertension`, `heart_disease`, and `stroke` are stored as integers even though they only ever hold 0 or 1.

A few numbers from the summary statistics stand out. Age ranges from 0.08 years (about a month old) to 82 years, average glucose level ranges from 55.12 to 271.74 mg/dL, and BMI ranges from 10.3 to 97.6 — the last of these is almost certainly not all real measurements, which is picked up again in Section 3.3. On gender, the dataset skews female: 2,994 patients (58.6%) are Female, 2,115 (41.4%) are Male, and exactly one row is labeled Other.

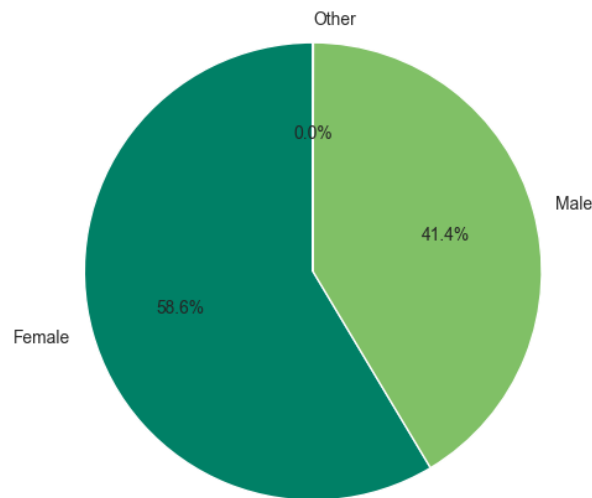


Figure 2: Gender distribution in the dataset

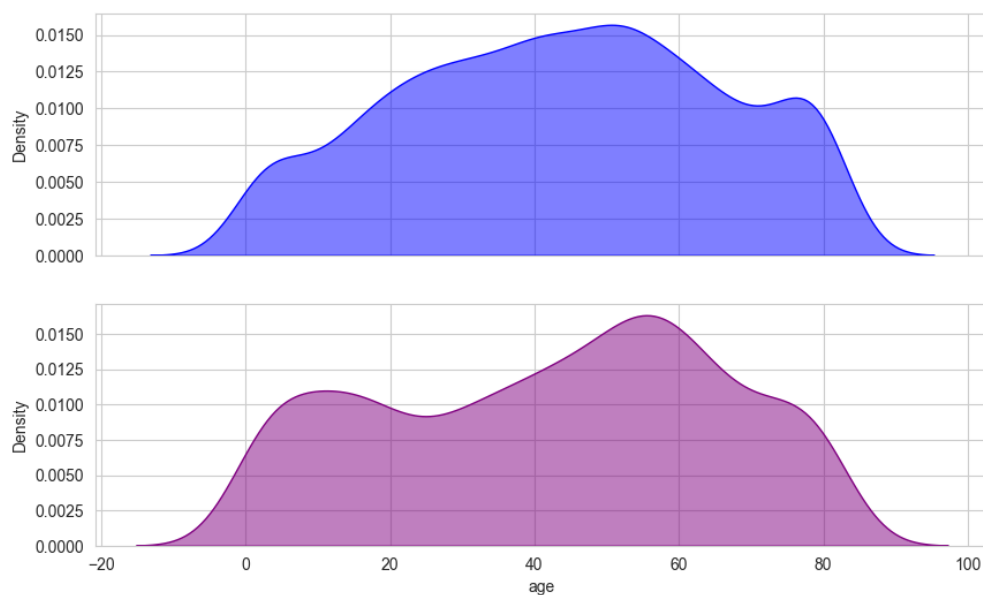


Figure 3: Age distribution by gender

The class distribution is where the real story is: only **249 of the 5,110 patients (4.9%) have had a stroke**, against 4,861 who have not. This kind of imbalance means a model that simply predicts “no stroke” for everyone would already be about 95% accurate while being clinically useless — a point that comes back repeatedly in Sections 3.6 and 3.7.

3.3 Data Cleaning and Preprocessing

The cleaning process followed the standard data-quality ordering — completeness, tidiness, validity, accuracy, then consistency — checking each dimension before moving to the next.

Completeness: Only `bmi` has missing values: 201 rows, or 3.93% of the dataset. BMI’s distribution is right-skewed (Figure 4), so the missing values were filled with the **median** rather than the mean, which avoids letting a handful of very high BMI patients drag the imputed value upward. Dropping those 201 rows instead was considered and rejected, since it would have removed close to 4% of the data along with some already-scarce stroke-positive cases.

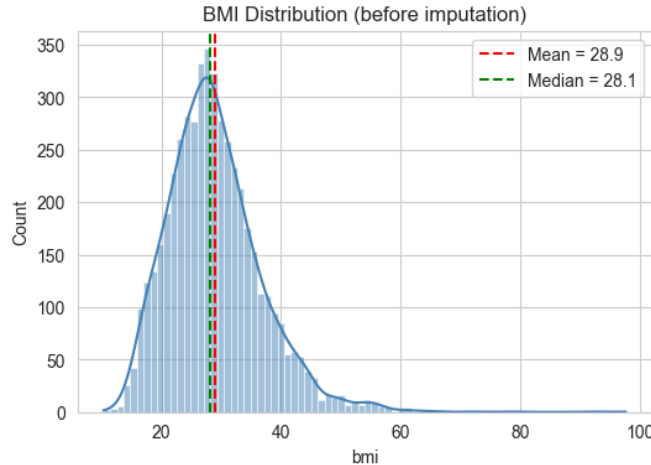


Figure 4: BMI distribution before imputation, with mean and median marked

There is a second, less obvious completeness issue: about 30.22% of `smoking_status` values are recorded as “Unknown.” This isn’t a blank cell, so `isnull()` doesn’t catch it, but it functions the same way — the smoking history simply wasn’t recorded for that patient. At 30% of the data, imputing it to the most common category would badly distort the feature, and dropping a third of the dataset isn’t realistic either. **Unknown** was kept as its own category and carried through to one-hot encoding, on the reasoning that not knowing a patient’s smoking history might itself correlate with other things (e.g., age — children are less likely to have a recorded smoking status in the first place).

Tidiness: `hypertension`, `heart_disease`, and `stroke` are binary flags but were stored as `int64`, which is more memory than a 0/1 flag needs. All three were downcast to `int8`.

Validity: No validity issues were found (e.g., no negative ages, no duplicate patient IDs), so this step was skipped.

Accuracy: Checking `age`, `avg_glucose_level`, and `bmi` with the IQR method (Figure 5) turned up very different pictures for each column:

- `age`: no outliers — the full 0–100 range sits comfortably within the IQR bounds.
- `avg_glucose_level`: 627 rows (12.3%) flagged, almost all of them patients with high glucose readings above roughly 169 mg/dL. These are not errors — they’re diabetic and prediabetic patients, and given that elevated glucose is a known stroke risk factor, removing them would have thrown away exactly the signal the model needs. These were left untouched.
- `bmi`: 110 rows (2.2%) flagged by the IQR rule, but most of these (BMI in the 40s–50s) are still medically plausible cases of severe obesity. Only the genuinely implausible values — BMI above 60, up to the recorded maximum of 97.6 — were capped down to 60. This affected 13 rows.

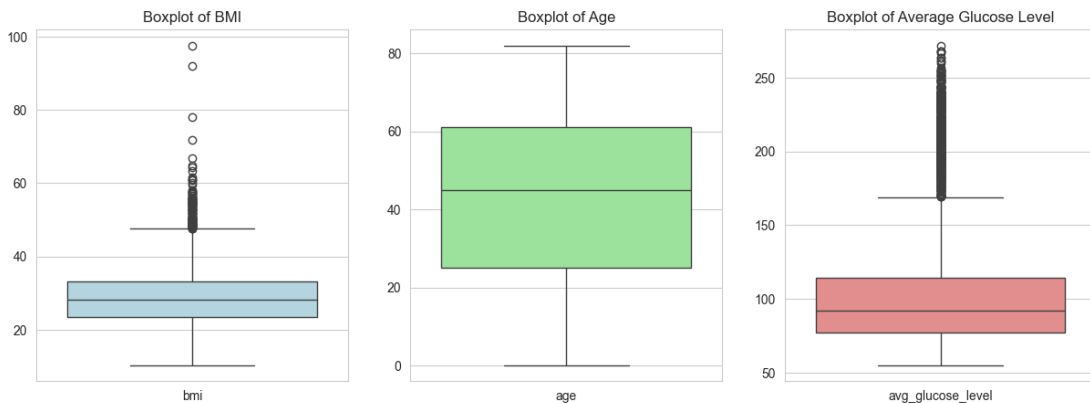


Figure 5: Boxplots of BMI, age, and average glucose level before cleaning

Consistency: No inconsistent labeling (e.g., mismatched capitalization or abbreviations) was found across categorical columns, so this step was also skipped.

No duplicate rows exist anywhere in the dataset. The cleaned data was written out separately so the exploratory analysis, feature engineering, and modeling stages that follow all build on the same consistent version.

3.4 Exploratory Data Analysis

Three charts were used to check whether the relationships the problem statement assumes — age, comorbidities, and glucose driving stroke risk — actually hold up

in the data.

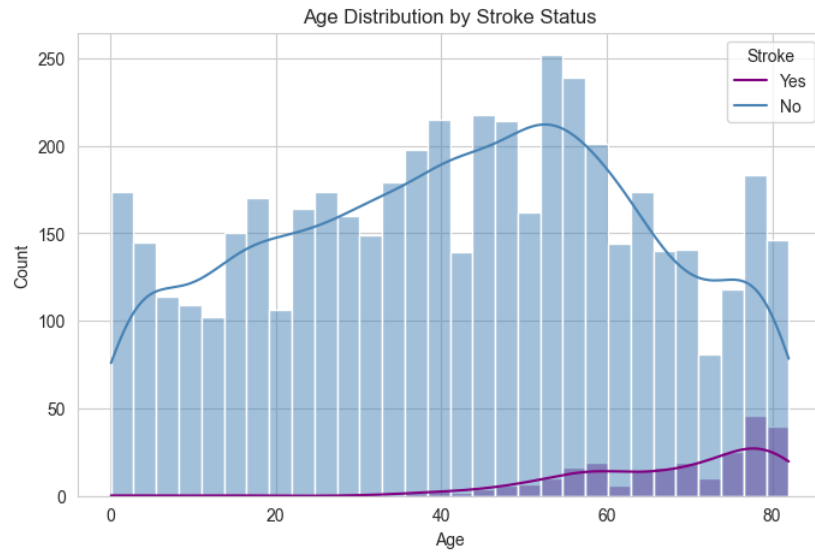


Figure 6: Age distribution split by stroke status

Age distribution by stroke status. Patients without a stroke are spread fairly evenly across the age range, but stroke cases cluster tightly above age 50 and are almost nonexistent below 30 (Figure 6). Age is clearly one of the strongest predictors available in this dataset, which lines up with the general medical understanding that blood vessels stiffen and accumulate damage over decades.

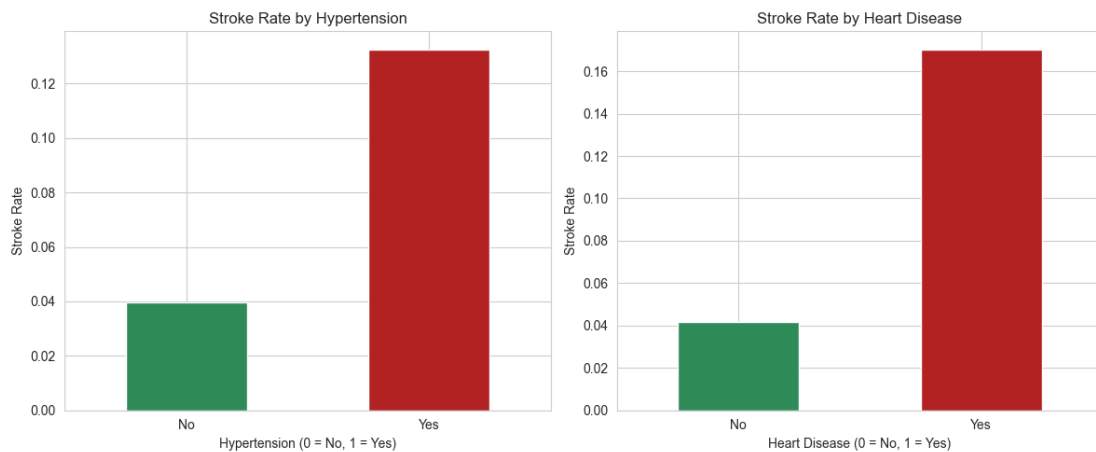


Figure 7: Stroke rate by hypertension and heart disease status

Stroke rate by hypertension and heart disease. Patients with hypertension or heart disease have a stroke rate roughly three to four times higher than patients without either condition (Figure 7). Both are established risk factors in clinical literature, and on their own — before even factoring in age — they already separate the data into a clearly higher-risk and lower-risk group.

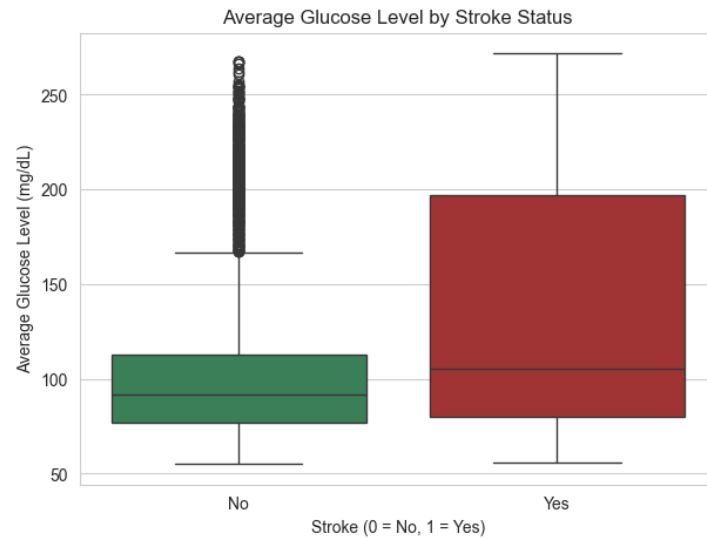


Figure 8: Average glucose level by stroke status

Average glucose level by stroke status. Stroke patients have a visibly higher median glucose level, with the spread reaching well into the 200+ mg/dL diabetic range, while non-stroke patients sit closer to normal levels (Figure 8). This also retroactively justifies the cleaning decision in Section 3.3 to leave the high-glucose readings in the data rather than treating them as outliers — they’re precisely the patients the model needs to learn from.

3.5 Feature Engineering

Five new features were added on top of the cleaned columns, each grounded in how these risk factors are actually interpreted clinically rather than as raw numbers (Table 5).

Table 5: Engineered Features and Rationale

Feature	Rationale
age_group	Bins age into five clinically meaningful ranges (Child, Young Adult, Adult, Middle Aged, Senior), capturing the non-linear jump in risk after 50 that a single linear <code>age</code> term can't fully express.
bmi_category	Groups BMI into the WHO-standard Underweight / Normal / Overweight / Obese bands, which is how BMI is actually read in practice rather than as a raw continuous number.
glucose_category	Groups average glucose level into Normal / Prediabetic / Diabetic using standard clinical cutoffs (140 and 199 mg/dL).
comorbidity_score	Simple sum of <code>hypertension</code> + <code>heart_disease</code> (0, 1, or 2), capturing that having both conditions together carries more risk than either alone.
age_hypertension	Interaction term ($\text{age} \times \text{hypertension}$). Hypertension matters more in an older patient than a younger one, and this term lets the model pick up on that directly instead of inferring it indirectly from two separate columns.

After creating these features, all categorical columns — the original five plus the three newly created ones (`age_group`, `bmi_category`, `glucose_category`) — were one-hot encoded, with the first category of each dropped to avoid redundant columns. This took the dataset from 17 columns to 29. The two binary flags already in 0/1 form (`hypertension`, `heart_disease`) were left as-is rather than re-encoded.

3.6 Machine Learning Model Development

With the target confirmed as binary and the positive class sitting at under 5% of the data, class imbalance had to be addressed directly rather than left to the models to sort out on their own — otherwise every model would gravitate toward simply predicting the majority class.

The data was split 80/20 into training and test sets, stratified on `stroke` so both sets kept roughly the same 4.9% stroke rate (4,088 training rows, 1,022 test rows; 4.87% and 4.89% stroke rate respectively). The three continuous columns most sensitive to scale — `age`, `avg_glucose_level`, `bmi` — along with the two engi-

neered numeric features `comorbidity_score` and `age_hypertension`, were standardized using `StandardScaler`, fit on the training data only and then applied to the test data to avoid leaking test-set statistics into training.

SMOTE (Synthetic Minority Over-sampling Technique) was then applied to the training set to balance the classes — going from 3,889 non-stroke vs. 199 stroke rows to 3,889 vs. 3,889. Critically, SMOTE was applied only to the training set; the test set was left in its original, real-world imbalanced state, since evaluating on artificially balanced data would give an overly optimistic picture of how the model performs in practice.

Three models were trained on the resampled, scaled training data:

1. **Logistic Regression** — a simple linear baseline whose coefficients are directly readable as risk direction and strength.
2. **Random Forest** (200 trees) — an ensemble that can pick up non-linear relationships and feature interactions without needing them to be manually specified.
3. **Gradient Boosting Classifier** — typically a strong performer on structured, tabular data of this kind.

3.7 Model Evaluation

Each model was evaluated on the held-out test set — which was never touched by SMOTE and reflects the real 4.9% stroke prevalence — using accuracy, precision, recall, F1-score, and ROC-AUC. Table 6 shows the results at the models’ default 0.5 classification threshold.

Table 6: Model Performance on the Test Set (default 0.5 threshold)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Gradient Boosting	0.9296	0.1563	0.10	0.1220	0.8031
Random Forest	0.9462	0.2727	0.06	0.0984	0.7953
Logistic Regression	0.0802	0.0505	1.00	0.0962	0.6399

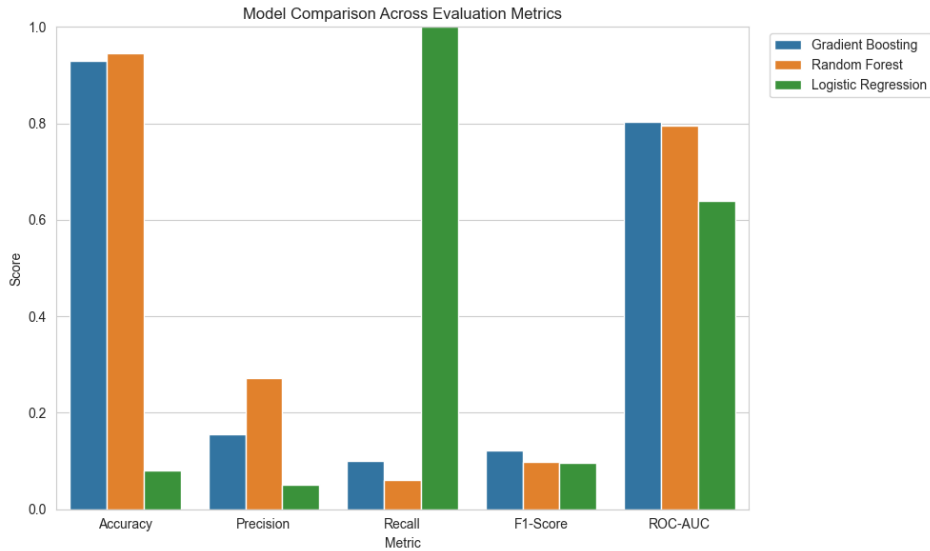


Figure 9: Model comparison across evaluation metrics

The Logistic Regression row needs unpacking, because on its own it looks almost broken: 100% recall, but only 8% accuracy and 5% precision. What’s happening is that Logistic Regression was trained on the SMOTE-balanced data (50% stroke, 50% no-stroke) and then evaluated at the standard 0.5 probability cutoff against the real test set, where stroke is actually only 4.9% of cases. A linear model trained on artificially balanced classes shifts its decision boundary to match that 50/50 split, so at test time it ends up flagging the overwhelming majority of patients — stroke or not — as high risk. It technically catches every real stroke case, but at the cost of drowning that signal in false alarms. Random Forest and Gradient Boosting are less sensitive to this shift and stay closer to sensible, if conservative, predictions, though both end up with low recall (6–10%) at this threshold — meaning they miss most actual stroke cases too, just for the opposite reason.

Because accuracy, precision, and recall all depend on where that 0.5 cutoff happens to fall, ROC-AUC — which measures how well a model ranks stroke patients above non-stroke patients regardless of threshold — gives a fairer read on which model is actually learning the underlying pattern. By that measure, **Gradient Boosting comes out ahead (0.8031)**, with Random Forest close behind (0.7953) and Logistic Regression noticeably lower (0.6399).

As a sanity check, 5-fold stratified cross-validation was also run on the original, non-resampled training data (no SMOTE), scoring purely on ROC-AUC:

Table 7: 5-Fold Cross-Validation ROC-AUC (original, non-resampled training data)

Model	Mean ROC-AUC	Std. Dev.
Logistic Regression	0.8378	0.0065
Gradient Boosting	0.8369	0.0174
Random Forest	0.8228	0.0151

Without SMOTE in the loop, all three models actually rank stroke risk quite well and land within a few points of each other — including Logistic Regression, which looked the worst in Table 6. That gap between the two evaluations is itself a finding: SMOTE improved nothing about how well these models can distinguish stroke from non-stroke patients (the underlying ranking ability was already there), but it did badly miscalibrate Logistic Regression’s decision threshold. Based on the test-set results at the deployed threshold, **Gradient Boosting is the best-performing model of the three**, though none of them reach a recall level that would be usable as-is in a real screening tool.

3.8 Discussion and Conclusion

The goal of this component was to predict stroke risk from a patient’s clinical and demographic profile, and the exploratory analysis in Section 3.4 confirmed that the dataset does carry the signal you’d expect — age, hypertension, heart disease, and glucose level all show a clear relationship with stroke, which the engineered features in Section 3.5 were built to capture more directly. Of the three models tested, Gradient Boosting gave the best balance of ranking ability and stable predictions on the real, imbalanced test data.

The main limitation surfaced in Section 3.7: at the default 0.5 threshold, none of the three models achieve a recall high enough to trust for actual screening, and SMOTE’s effect on Logistic Regression in particular shows that oversampling the training data can distort a model’s decision boundary even when it doesn’t touch its underlying discriminative ability. A model that misses 90% of real stroke cases (as Gradient Boosting does here) is not something a clinic could rely on, whatever its ROC-AUC looks like. The dataset’s severe class imbalance — only 249 stroke cases out of 5,110 patients — makes this a genuinely hard problem to model well with a small feature set, and three baseline models with default hyperparameters is a reasonable starting point rather than a finished solution.

A few directions would likely improve this: tuning the classification threshold (or calibrating predicted probabilities) instead of defaulting to 0.5, especially for the SMOTE-trained models; trying `class_weight='balanced'` as an alternative

to oversampling, since it doesn't distort the training distribution the same way; running a proper hyperparameter search (grid or random search) rather than default settings for Random Forest and Gradient Boosting; and testing a few additional model families, such as XGBoost or a support vector machine, since the cross-validation results suggest there's more separable signal in this data than the current threshold-based results reflect.

References

1. Task 1 LP_Solver Excel Sheet – Google Drive
2. Task 2 Network Diagram – Github
3. Task 3 Notebook – Google Colab.